Grading Rubrics

Dimension	Sub- Dimension	Notes for Auditors	2	3	5
Prompt	**Prompt Clarity and Specificity**	The Prompt is not required to cover every single line of code changed in the Golden Patch (e.g., unrelated documentation fixes or minor refactoring). The Prompt must only comprehensively cover the functionality explicitly tested by the Fail-to-Pass (F2P) test cases.	**[Fail - Major Clarity / Specificity Issues]** - It's not clear what is being asked, the prompt is extremely difficult to follow, or is overly vague; the user intent cannot be reasonably understood from the prompt. - Major details are missing that are needed to answer the prompt and cannot be reasonably assumed	**[Non-Fail - Minor Clarity / Specificity Issues]** - It's mostly clear what is being asked but the request could reasonably be interpreted multiple ways	- There is little to no room for misinterpretation of the specific request - Prompt has a specific request that doesn't require more than one minor assumption to answer it
Prompt	**Domain Relevance**		**[Fail - Task Relevance]** - The prompt contains egregious misalignment regarding the Git Hub Repository URL, provided commits or tool files.	**[Non-Fail - Task Relevance]** - The prompt contains minor misalignment regarding the Git Hub Repository URL, provided commits or tool files.	- The Prompt is clearly related to the assigned task parameters.
Prompt	**Category Relevance**		**[Fail - Category Relevance]** - The Prompt does not relate to any of the 3 task types: Repo Evolution, Multi-Component Feature, or Tool Implementation.	**[Non-Fail - Category Relevance]** - The Prompt has an egregious misalignment with the assigned task type, but it is still relevant to one of the 3 task types of the project. **NOTE**: Please note in the feedback which task type you believe best applies to the Prompt.	- The Prompt clearly fits the assigned task category.
Prompt	**Self-Containment [Tool Implementation Only]**	*[Only applicable to Tool Implementation tasks]* We should assume that the prompt does not have access to the repo. (If there is any repo context required to create the tool, for whatever reason, it should be included with the prompt.) As an example, a CB may pull the basis of their proposed tool—the seed idea and code— from the provided PR, but their prompt should not require knowledge of the repo's internal workings to fulfill. The resultant tool should be able to be applied to any repo; the code should be modified to be repo-independent. (As such, the test patch should cover the functionality and edge cases of the tool rather than its integration with the assigned repo.) 'Self-contained' means the tool must not depend on the specific internal workings, databases, or proprietary modules of the provided GitHub Repository. It does not mean the tool must be a single file. A valid Tool Implementation can, and often should, consist of multiple scripts/modules (e.g., an entry point and several helper modules). If the Golden Patch includes multiple modules that form a standalone tool, this is perfectly valid and should not be penalized.	**[Fail - Prompt Not Self-Contained]** - The prompt's main requests cannot be fulfilled without access to the repo, except entry point + modules. (All necessary context should be provided by the prompt.)	**[Non-Fail - Prompt Not Self-Contained]** - The main request is fully self-contained within the prompt; however, certain assumptions are required that do not affect the overall behaviour and functionality of the tool.	- The prompt is fully self-contained, and no assumptions are needed to fulfill the prompt.
Prompt	**Interfaces**	The interface should describe the **minimal external interaction** point that	**[Fail - Missing Interface Section]** The interface	**[Non Fail - Invalid or Undocumented Interface]**	- All required interfaces are properly documented

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-13T04:19:09.292Z

		evaluation scripts (i.e., **F2P test cases** in practice) use to interact with the solution.
		It must:
		• Define the natural entry point (function, class, CLI command, or API endpoint)
		• Specify inputs and outputs with clear typing
		• Remain implementation agnostic (no internal architecture details)
		• The interface should only include elements required to evaluate the task.
		
		NOTE: For **Repo Evolution** and **Multi-Component** tasks, it is fine for the prompt to be detailed and include more implementation-specific details, than just the **minimal external interaction**. This should not be penalized
		
		Each interface entry should be assessed for whether it correctly defines the external contract. Interfaces should be classified into:
		Valid Interface
		The interface correctly defines:
		• The entry point
		• Input arguments and types
		• Output format or return type
		• It does not reference internal implementation details.
		
		Interfaces with minor inaccuracy
		The interface generally represents the correct contract but contains small issues such as:
		• Slight type imprecision (ex: INT vs LONG)
		• Minor documentation inconsistencies that do not hinder valid usage of the interface
		  ○ For example, inconsistent descriptions
		  ○ Insignificant typos like "Nam: correct_interface_name" where "Nam" doesn't hinder usage of the interface
		• Missing descriptions
		• Interfaces that are not required for interacting with the F2P test suite. For example, interfaces documenting internal helper functions, interfaces that are strictly concerning P2P test suite, etc
		• These issues do not prevent evaluation scripts from using the interface correctly.
		
		Invalid Interface / Missing Interfaces
		The interface fails to correctly represent the contract required to evaluate the solution, or an interface used by the F2P test suite is entirely missing.
		Examples:
		• Missing entry point
		• Missing contract fields - path/name/type/input/output
		• Incorrect interface signature (wrong path/name/type/input/output) other than the ones excluded under "minor inaccuracies" above
		
		CB can focus on changes ONLY rather than documenting the entire interface (as the previous state is known to the model). For example, in an interface with 5 arguments, if only one of them changed, only that argument needs to be mentioned in the interface and the rest can be omitted.
			section is missing entirely or is empty
			
			[Fail - Invalid or Undocumented Interface]
			More than 10% of interfaces are **Invalid / Missing** and fail to correctly define the external contract or the introduced changes.
			
			[Fail - Major Interface Accuracy Issues]
			More than 20% of the interfaces have **minor or major inaccuracies combined**.
			
			% = (i + m) / t * 100
			
			Where:
			• i = number of interfaces documented by the CB with an issue
			  ○ do not double count issues
			  ○ if an interface has both major and minor issues, only count that as a major issue
			• m = number of missing F2P related interfaces
			• t = correct number of total interfaces after account for missing interfaces
				10% (or less) of interfaces are Invalid and fail to correctly define the external contract or the introduced changes.
				
				[Non Fail - Minor Interface Accuracy Issues]
				20% (or less) of the interfaces have minor and major inaccuracies combined.
				
				% = (i + m) / t * 100	

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-13T04:19:09.292Z

Uploaded Files	**Presence**		**[Fail - Missing Critical Files]** - Any of the following files are missing: - "Before" JSON output - "After" JSON output - Run script - Parsing script - Dockerfile - Golden Patch		- All required files are correctly included in the task.
Golden Patch - Repo Evolution **(Applicable ONLY in "Repo Evolution" tasks)**	**Alignment with Repo Patch**	Contributors do not change the golden patch in 'Repo Evolution' tasks. QC should only assess that the Golden Patch is the same in the repo and in the task – or is a valid cutoff of it. A valid cutoff is a 'shorter' version of the golden patch (which could originally have like thousands of lines) with only the relevant code for the prompt request / expected interfaces to use.	**[Fail - Golden Patch Mismatch]** - (Repo Evolution tasks only) The golden patch in the task does not match the patch in the repo or is not a valid cutoff	N/A	Golden patch in the task matches the repo patch or is a valid cutoff
Golden Patch **(Applicable in "Repo Evolution" and "Multi-component " tasks)**	**Fulfillment Quick Check**	"Fulfillment" under this dimension is defined as whether the instructions are *attempted* - regardless of whether they are correctly executed. As long as there is indication that all the instructions were acknowledged / registered in the changes, we can consider that the golden patch fulfills the prompt	**[Fail - Unfulfilling Golden Patch]** - At least one instruction is completely ignored in the golden patch	N/A	All the instructions are at least *attempted*
Golden Patch **(Applicable only in "Tool Implementation" tasks -**	**Validity**	**NOTE**: Make sure to inspect the Dockerfile if you have issues applying patches—it may provide clues as to what's going on. (e.g., the Dockerfile could be cloning the wrong commit.) The Prompt is not required to cover every single line of code changed in the Golden Patch (e.g., unrelated documentation fixes or minor refactoring). The Prompt must only comprehensively cover the functionality explicitly tested by the Fail-to-Pass (F2P) test cases.	**[Fail - Golden Patch Incorrect]** - Any of the prompt's explicit requests are not fulfilled. Note: do not penalize anything related to the design itself. **[Fail - Golden Patch Invalid]** - The Golden Patch has compilation or execution issues.		- The Golden Patch does not have any compilation or execution issues. - The Golden Patch implements all aspects of the prompt.
Golden Patch **(Applicable only in "Tool Implementation" tasks -)**	**Code Design**		**[Fail - Major Code Design Issues]** - Code implementation uses poor design choices and lacks even the fundamental principles such as modularity and abstraction. It would be hard to maintain and extend this code down the line	**[Non-Fail - Minor Code Design Issues]** - Code implementation partially adheres to design principles (bare minimum) but there is room for improvement	- Code implementation follows most of the design principles, making the code easy to maintain and extend down the line (Code design constitutes good programming practices such as modularity, separation of concerns, abstraction, etc)
Golden Patch **(Applicable only in "Tool Implementation" tasks)**	**Performance**		**[Fail - Major Code Performance Issues]** - Code implementation is highly inefficient with clear room for significant efficient improvement Ex 1: Takes an O(n^3) brute force approach when it's possible to fulfill the request in O(n log n)	**[Non-Fail - Minor Code Performance Issues]** - The code implementation is moderately efficient with room for further optimization. Ex: O(n^2) was used when O(n log n) is possible. This is allowable unless it goes against specific requests from the prompt.	- The code implementation is well-optimized, utilizing efficient algorithms and data structures wherever possible

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-13T04:19:09.292Z

Golden Patch **(Applicable only in "Tool Implementation" tasks)**	**Readability**		**[Fail - Major Code Readability Issues]** - The code is difficult to read in > 2 areas because of poor formatting such as missing indentation, poor markdown, excessive white space, or no whitespace (minified code) **[Fail - Misleading Code Variable Names]** - Variable/class/method names are not indicative of their function. Ex: a misleading variable name such as `even_array = [1, 3, 5, 7]`	**[Non- Fail - Minor Code Readability Issues]** - The code can be formatted better in <= 2 areas (because of poor formatting such as missing indentation, poor markdown, excessive white space, or no whitespace (minified code)), but it's still readable - Code has egregious formatting changes but it does not affect the function of the code **[Non-Fail - Poor Code Variable Names]** - Variable/class/method names don't follow the general naming conventions of the respective language	- The code is well organized and uses consistent formatting, making it highly readable - Variable/class/method names are meaningfully chosen and are reflective of their purpose
Golden Patch **(Applicable only in "Tool Implementation" tasks)**	**Code Safety/Security**		**[Fail - Unsafe Code]** - The response code clearly has a malicious intention (e.g., virus/malware generation, exposes personal information) - The code implementation can introduce common security vulnerabilities, such as SQL Injection	N/A	- The code implementation has no security or safety issues
Tests **(Not applicable to "Repo Evolution" Tasks, see "Golden Patch - Repo Evolution")**	**Coverage**	This can be evaluated by exploring the provided test patch. It is acceptable for the Golden Patch to contain extra, untested code, as long as the F2P tests fully cover the requirements defined in the Prompt. Do not penalize if the Golden Patch includes changes not explicitly asked for, provided they do not break Pass-to-Pass (P2P) tests. **NOTE**: While the core process of validating the Golden Patch remains the same, the nature of the tests will differ significantly between fixing a bug in an existing repo and building a new tool from scratch ("Tool Implementation"). The test suite for "Tool Implementation" tasks should revolve primarily around tool functionality and edge cases rather than integration with the assigned repo.	**[Fail - Missing Test] - Full tool implementation only** - At least one essential test—required to verify an explicit request from the prompt or ensure functional code—is missing from the test suite. **[Fail - Missing Test] - Not Full tool implementation** - The F2P test does not cover the core, complex functionalities explicitly defined in the Prompt.		- All essential tests are covered within the test suite.
Tests **(Applicable only in "Tool Implementation" tasks)**	**Relevance**	This can be evaluated by exploring the provided test patch. **NOTE**: While the core process of validating the Golden Patch remains the same, the nature of the tests will differ significantly between fixing a bug in an existing repo and building a new tool from scratch ("Tool Implementation"). The test suite for "Tool Implementation" tasks should revolve primarily around tool functionality and edge cases rather than integration with the assigned repo. Tests should be classified into (according to relevance): **Relevant** The test verifies behavior required or reasonably implied by the prompt through the Expected Interface. **Weakly Relevant** The test checks behavior that is related to the interface but **not clearly required**	**[Fail - over 10% Irrelevant Tests]** - More than 10% of tests are irrelevant to the prompt's explicit and implicit requests. **[Fail - over 20% Weakly Relevant Tests]** - More than 20% of tests are weakly relevant or irrelevant to the prompt's explicit and implicit requests.	**[Non Fail - 10% or less Irrelevant Tests]** - 10% (or less) of tests are irrelevant to the prompt's explicit and implicit requests. **[Non Fail - 20% or less Weakly Relevant Tests]** - 20% (or less) of tests are weakly relevant or irrelevant to the prompt's explicit and implicit requests.	- All tests are relevant to the prompt's explicit and implicit requests.

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-13T04:19:09.292Z

		or **implied** by the prompt. These tests are acceptable but may introduce noise. Examples: • Overly strict formatting • Minor behavioral constraints not mentioned in the prompt • Additional edge cases that are unnecessary but still related **Irrelevant** The test verifies behavior **unrelated** to the prompt's explicit or implicit requirements. Examples: • Testing unrelated functionality • Testing internal implementation details • Mocking or patching internal components • Verifying behavior not exposed through the Expected Interface			
Before and After JSON	**Validity**	This can largely be evaluated by comparing the CB-provided JSON logs to the JSON logs generated during the Docker workflow (see Docker Workflow Guide)—they should be identical.	**[Fail - Invalid JSON]** - Any of the required JSON files—Before and After—are missing - Any of the provided JSON files are syntactically invalid, irrelevant, or otherwise incorrect		- All required JSON files are present. - All provided JSON files are syntactically valid, relevant, and correct.
Before and After JSON	**Output**	The "After" JSON log represents the output of the test suite evaluated against the Golden Patch. All F2P tests should Pass, and there should be no P2F (Pass-to-Fail) tests. **NOTE**: Sometimes, the "Before" log will show failures for "test cases"—groupings of assertions that test a cohesive intent—while the "After" log will list all of the individual assertions belonging to these test cases. Please keep this in mind before penalizing for misalignment.	**[Fail - Failing Tests in After JSON]** - At least one of the F2P tests in the "After" JSON log returns as "FAILED" (or semantic equivalent). - At least one test which "PASSED" (in the "Before" JSON log) now returns as "FAILED" (or semantic equivalent) in the "After" JSON log.		- All of the F2P tests in the "After" JSON log returns as "PASSED" (or semantic equivalent). - All of the tests which passed in the "Before" JSON log also pass in the "After" JSON log.
Run Script	**Relevance**	NOTE: Don't penalize redundancies as "irrelevant".	**[Fail - Run Script Irrelevant]** - The Run Script has irrelevant code.		- The Run Script is fully relevant to the task.
Parsing Script	**Relevance**	NOTE: Don't penalize redundancies as "irrelevant".	**[Fail - Parsing Script Irrelevant]** - The Parsing Script has irrelevant code.		- The Parsing Script is fully relevant to the task.
Dockerfile	**Dockerfile**	The dockerfile should not have any build or runtime errors. There is a script that automatically catches any dockerfile errors, so this dimension is for catching any missed instances. Make sure to build the Dockerfile using the `--platform linux/amd64` tag **Note**: The dockerfile's extension will be sanitized - do NOT penalize for the inclusion of an extension. Currently, the platform doesn't allow CBs to upload the file's MIME type so CBs might use .txt or something similar.	**[Fail - Invalid Dockerfile]** The dockerfile does not run or build as-is	N/A	The dockerfile runs/builds correctly
Dockerfile	**Relevance**	NOTE: Don't penalize redundancies as "irrelevant".	**[Fail - Dockerfile Irrelevant]**		- The Dockerfile is fully relevant to the task.

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-13T04:19:09.292Z

			- The Dockerfile has irrelevant code.		
Rubrics	**Overall Rubric Quality** **[Only Tool Implementation]**	Issues with rubric criteria are divided into three types - Major, Moderate and Minor. All the errors across rubrics are tallied at the end to arrive at a holistic rating for the entire rubric. See the next columns to understand the thresholds. **See Rubric Quality Definitions section below to understand what counts as a Major, Moderate and Minor issue.** Use the number of criteria that the CB wrote as the denominator while calculating % values. Numerator is the number **of criteria with issues of the type** (see definition below) among criteria. Take note that some issues have bespoke counting rules (mentioned under "Definition"). **Do NOT double count criteria while tallying even if it has multiple issues. If a criteria has 1 major issue and 1 moderate issue, count it as having 1 major issue only.**	**Use the number of criteria that the CB wrote as the denominator while calculating % values. See the additional notes section for the numerator. Do NOT double count criteria while tallying even if it has multiple issues.** **[Fail - 10%+ Major Rubric Errors]** - More than 10% of the criteria contain major issues **[Fail - 25%+ Moderate Rubric Errors]** - More than 25% of the criteria contain moderate or major issues **[Fail - 35%+ Minor Rubric Errors]** - More than 35% of the criteria contain minor or moderate or major issues	**Use the number of criteria that the CB wrote as the denominator while calculating % values. See the additional notes section for the numerator. Do NOT double count criteria while tallying even if it has multiple issues.** **[Non-Fail - Up to 10% Major Errors]** - Up to 10% (<=10%) of the criteria contain major issues **[Non-Fail - Up to 25% Moderate Errors]** - Up to 25% (<=25%) of criteria contain moderate or major issues (with major issues contributing lower than 10%) **[Non-Fail - 10-35% Minor Errors]** - Between 10 and 35% (>=10% and <=35%) of criteria contain minor or moderate or major issues (with major issues contributing lower than 10% and moderate issues contributing lower than 25%)	- Less than 10% (<10%) of the rubrics have minor issues - No major or moderate issues
All CB Generated Content	**Native Fluency**		**[Fail - Lacks Native Fluency]** - Writing is not in English.	**[Non-Fail - Minor Fluency Errors]** - Writing is mostly that of an English speaker, but there is a strange phrase that gives you pause.	- The writing contains native-level English.
All CB Generated Content	**Spelling / Grammar / Formatting**		**[Fail - SGP Major Error]** - Has 4 or more spelling (minor), grammar, and punctuation errors **[Fail - Multiple Egregious Spelling Errors]** - Has 2 or more egregious spelling errors **[Fail - Major Formatting Errors]** - The response contains broken formatting such as broken formatting for a list or broken markdown Note: For spelling, grammar, and punctuation, errors accumulate across the entire task. For example, if the prompt had 2 spelling errors and the response had 2	**[Non-Fail - SGP Minor Error]** - Has up to 3 spelling, grammar and punctuation errors **[Non-Fail - Minor Egregious Spelling Error]** - Has 1 egregious spelling error **[Non- Fail - Minor Formatting Error]** - Minor formatting issues such as multiple new lines between content	- There are no easily discernible errors

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-13T04:19:09.292Z

			spelling errors, we'd fail the task		
All CB Generated Content	**Harmful Content**		**[Fail - Harmful Content]** - The material contains any harmful content and the project does not call for it	N/A	- Native-level writing for English
All CB Generated Content	**Un-auditable**		**- [Un-auditable]** - Used for auditors to skip tasks when the system otherwise does not allow. Error Category must be exactly configured as "Un-auditable" to work with QC query logic. To be used as a last resort.	N/A	
All CB Generated Content	**Unlisted Minor Errors**		N/A	**[Non-Fail - Unlisted Minor Errors]** - There are errors in the task that are not explicitly mentioned in the grading rubrics but prevent the task from being perfect.	

Rubric Quality Definitions - Only Applicable to Tool Implementation

Major Issues	**Missing Criteria - Critical Requirements**	**Definition:** • Count each missing rubric that ought to check for an explicit requirement in the prompt or a critical implicit expectation of the prompt as one issue (critical = you cannot imagine a good response without it) NOTE: The purpose of the rubrics is not to exhaustively cover every possible item or behavior implied by the prompt. As defined in the project taxonomy and stated in the overall project goal, rubrics are intended to evaluate aspects of the response that cannot be deterministically validated through automated tests or to complement them. In other words, rubrics are designed to complement the test suite, not to replace it. A correct evaluation must consider the combined outcome of both mechanisms: automated tests and rubric-based assessment. This is not an either-or approach, but a joint evaluation framework. Do not flag a missing rubric if a unit test is already covering the same thing.
	Criteria Not Self Contained	**Definition:** Criterion cannot be evaluated against the model response without access to the prompt, reference text, other criteria, and/or external facts/information Every rubric must be self-contained. Imagine that you only have access to the model response and are trying to evaluate if it fulfilled the rubric item. Will you be able to evaluate accurately without referencing anything else? Some criteria are self-explanatory but often, this translates to the criteria mentioning the answer to the prompt directly. Examples of criteria that are not self-contained: • Example 1: "Response identifies the first president of the USA" ○ Fixed: "Response identifies the first president of the USA as George Washington" • Example 2: "The response addresses the bug mentioned in the prompt"

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-13T04:19:09.292Z

		○ Fixed: "The response addresses the bug where the submit button doesn't work"
	Criteria Not Atomic - Major	**Definition:** Criterion groups two or more constraints that are completely unrelated, which results in a rubric item with no clear focus on what aspect of the response it's trying to evaluate. These constraints cannot be interpreted as part of a single coherent instruction but reads more like a dump of requirements. Each rubric should evaluate one thing only — no bundling of multiple behaviors. Ask yourself if the criterion is evaluating more than one idea.
	Incorrect Criteria	**Definition:** • Criterion checks for something that does not align with prompt requirements • Criterion contains a factual error or a misleading point ○ Example: "The response implements a sorting algorithm that runs in O(nlogn), such as selection sort" • Criterion is not an explicit requirement in the prompt and implementing it does not make the response worse • Criterion is not at all related to the requests in the prompt **NOTE:** Before classifying any issue as "Incorrect criteria", see if a different, more specific error category would apply. For example, if a criterion is overly specific, you could argue that it's "incorrect", but it should still be counted as "Overfitting and Underfitting".
	Framing	**Definition:** • Criteria that are not framed in a way such that a good response would evaluate to "Yes" or "True" Criteria should be positively *framed*. Meaning, they should all evaluate to "True" or "Yes" or "Pass" for a good response. Incorrectly framed criteria are as bad as incorrect criteria.
Moderate Issues	**Missing Criteria — Non-critical Requirements**	**Definition:** • Count each missing rubric that ought to check for a non-critical explicit requirement or implicit expectation of the prompt as one issue (critical = you cannot imagine a good response without it) • Example non-critical requirements: "Use bold text", "Use bullet points" NOTE: The purpose of the rubrics is not to exhaustively cover every possible item or behavior implied by the prompt. As defined in the project taxonomy and stated in the overall project goal, rubrics are intended to evaluate aspects of the response that cannot be deterministically validated through automated tests or to complement them. In other words, rubrics are designed to complement the test suite, not to replace it. A correct evaluation must consider the combined outcome of both mechanisms: automated tests and rubric-based assessment. This is not an either-or approach, but a joint evaluation framework. Do not flag a missing rubric if a unit test is already covering the same thing.
	Overlapping or Redundant Criteria	**Definition:** • Criterion that is either completely redundant because other criteria completely encompass the former or multiple criteria that check for the same thing partly • Count each completely redundant criteria as one moderate issue or count multiple overlapping criteria as one moderate issue Redundant Scenario: Criteria 1: Response does a, b, c Criteria 2: Response does a, b

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-13T04:19:09.292Z

		Overlap Scenario: Criteria 1: Response does a, b, c Criteria 2: Response does b, c, d Note that this applies to cases where two criteria independently assess the same elements, not when a single criterion introduces and specifies related requirements. ("The response follows best code practices by ensuring that each line is under 79 characters" is acceptable.)
	Overfitting and Underfitting	**Definition:** • **Overfitting:** Criteria that are overly specific, inflexible or too rigid - they incorrectly reject a subset of valid implementations • **Underfitting:** Criteria that are overly broad, permissive or loose - they accept valid implementations, but also incorrectly accept invalid implementations too Criteria must be flexible enough to accept different valid implementations. Note that criteria can mention specific answers as long as they are provided as examples in any way. i.e. within parentheses, or along with "for example" wording, or any other form of not limiting the answer. Conversely, criteria must not be too permissive or overly broad, such that they would also accept invalid implementations along with valid implementations.
	Subjective Criteria	**Definition:** • Criteria that are subjective, vague or immeasurable (e.g., "the response should have good formatting" or "code must be optimal") A criterion should be evaluated based on whether its primary requirement is measurable, even if it includes additional context or reasoning that's less precise Using vague or subjective qualifiers like "appropriate", "properly", "best practices", "reasonable" etc without attaching explicit definitions makes criteria unmeasurable and should be flagged here. Subjectivity with certain details should be acceptable when the prompt is intentionally ambiguous and open-ended For example, for a prompt "Create an artistic website to showcase my sculptures, featuring an animated background that looks like shifting, fluid marble", the following rubric is acceptable: "The website has a refined modern look so it could be launched as a product by a reputable company."
	Incorrect Weights - Major	**Definition:** • Criteria that are objectively incorrectly weighted by two levels, i.e., 1 is selected when 5 is appropriate or vice versa Criteria are categorized into one of 3 weight buckets: 1 (low), 3 (medium), or 5 (high)
	Criteria Not Atomic - Minor	**Definition:** • Criterion groups two or more constraints that are only partially related. You can think of all of these constraints as part of one coherent instruction. Each rubric should evaluate one thing only — no bundling of multiple behaviors. Ask yourself if the criterion is evaluating more than one idea.
Minor Issues	**Incorrect Weights - Minor**	**Definition:** • Criteria that are objectively incorrectly weighted by one level. i.e., 1v3 or 3v5 scenarios Criteria are categorized into one of 3 weight buckets: 1 (nice-to-have), 3 (important), or 5 (critical)
	Miscategorized Criteria	**Definition:** Criteria are objectively tagged with the wrong category when there is a better one available. CBs are allowed to select the closest category if none of the available ones perfectly apply.

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-13T04:19:09.292Z

		<u>List of categories:</u> • Instruction Following: Ensures the response adheres to explicit directions in the prompt (format, constraints, language, libraries, required elements). • Code Correctness: Ensures any code performs the intended task and produces correct results based on the prompt. • Code Quality: Covers robustness, maintainability, idiomatic patterns, and avoiding fragile or error-prone design choices. • Code Clarity: Covers readable, well-structured code, including naming, organization, and formatting. • Code Efficiency: Covers conciseness, avoidance of unnecessary steps, and reduction of redundancy.

CONFIDENTIAL | latam.coder1936@remotasks.com+outlier | 2026-03-13T04:19:09.292Z